Apache Cassandra

Borrador del análisis de la Unidad 1

3. Análisis con los conceptos de la Unidad 1

3.1. CAP y PACELC

CAP. Si una partición de red impide que los nodos se comuniquen, el teorema CAP establece que el sistema no puede garantizar a la vez la consistencia linealizable y una respuesta exitosa a toda solicitud que llegue a un nodo operativo [1]. La prueba de Cassandra permite examinar una consecuencia relacionada. Se detuvieron dos contenedores, de modo que el coordinador solo tenía una réplica disponible. Esa réplica devolvió la fila con `ONE`. Con `QUORUM`, el servidor devolvió `Unavailable` porque necesitaba dos réplicas y solo tenía una activa. El resultado muestra cómo el mínimo de respuestas afecta la disponibilidad de una lectura.

Cassandra permite elegir el nivel de consistencia por operación [2]. Con `ONE`, en otra ejecución, una lectura podría obtener un valor desactualizado si la réplica consultada aún no recibió una escritura reciente. En esta prueba, la fila se insertó con `ALL` antes de detener los nodos y se leyó sin modificarla. La pérdida de acceso a dos réplicas sirve para ilustrar el dilema de CAP. Comprobar el comportamiento ante una partición real requeriría mantener los nodos activos y bloquear la comunicación entre ellos. Evaluar la consistencia linealizable requeriría además observar el orden de lecturas y escrituras.

PACELC. El modelo de Abadi plantea una elección entre disponibilidad y consistencia durante una partición (PAC), y entre latencia y consistencia cuando la red funciona con normalidad (ELC) [3]. Con los tres nodos activos, ambas lecturas devolvieron la fila. `QUORUM` debía esperar dos respuestas para completar la consulta, mientras que a `ONE` le bastaba una. Esto permite plantear que esperar una segunda respuesta puede aumentar la latencia, sobre todo si una réplica tarda en responder. Las capturas registran el éxito o fallo de la consulta y la hora de cada etapa. Medir la latencia requeriría cronometrar consultas repetidas bajo las mismas condiciones.

Como contraste, Redžepagić et al. midieron la latencia de *escritura* de Cassandra en Kubernetes. En la Tabla 22 de su estudio, el promedio pasó de 1,23 ms con `ONE` a 3,16 ms con `QUORUM` [4]. Esas cifras corresponden a la carga y al entorno de ese estudio. En el laboratorio de este informe, la comparación se limitó a la disponibilidad de una lectura.

3.2. Tolerancia a fallos

La replicación permitió leer la fila aun con dos nodos detenidos. `nodetool status` mostró dos nodos en `DN` (caídos, en estado normal dentro del anillo) y uno en `UN` (activo, en estado normal). Con factor de replicación tres, el umbral de `ONE` permite tolerar la pérdida de hasta dos réplicas para leer un dato presente en la réplica accesible. `QUORUM` requiere dos réplicas accesibles y, por tanto, admite la pérdida de una [2]. La prueba observó los extremos de tres y una réplica activa.

Tras reiniciar los contenedores, los tres nodos volvieron a `UN` y la consulta con `QUORUM` volvió a devolver la fila. Se recuperó así la capacidad de completar esa lectura con el umbral solicitado. Para comprobar la sincronización de las réplicas durante la recuperación, haría falta escribir un dato mientras los otros nodos están detenidos y revisar si lo reciben al volver. Además, los tres contenedores comparten una computadora. Si esta se apaga, los tres nodos quedan fuera de servicio al mismo tiempo.

Referencias

[1] S. Gilbert y N. Lynch, «Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002, doi: 10.1145/564585.564601.

[2] Apache Cassandra Project. «Dynamo.» Apache Cassandra Documentation, versión 5.0, visitado 12 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/dynamo.html

[3] D. J. Abadi, «Consistency Tradeoffs in Modern Distributed Database System Design: CAP Is Only Part of the Story,» *Computer*, vol. 45, n.º 2, págs. 37-42, 2012, doi: 10.1109/MC.2012.33.

[4] J. Redžepagić, A. Kapulica, N. Malešević y V. Dakić, «Empirical Performance and Operational Analysis of Monolithic and Distributed Database Architectures in Kubernetes Environments,» *Computers*, vol. 15, n.º 5, 2026, art. 282, doi: 10.3390/computers15050282.